

6

Connecting to a Database

In the last chapter, we looked at how to structure a FastAPI application. We successfully implemented some routes and models for our application and tested the endpoints. However, the application still uses an in-app database to store the events. In this chapter, we will migrate the application to use a proper database.

A database can be simply referred to as a storehouse for data. In this context, a database enables us to store data permanently, as opposed to an in-app database, which is wiped off upon any app restart or crash. **A database is a table housing columns, referred to as fields, and rows, referred to as records.**

By the end of this chapter, you will be equipped with the knowledge of how to connect a FastAPI application to a database. This chapter will explain how to connect to a SQL database using **SQLModel** and a **MongoDB** database via **Beanie**. (However, the application will make use of MongoDB as its primary database in later chapters.) In this chapter, you'll be covering the following topics:

- Setting up SQLModel
- CRUD operations on a SQL database using SQLModel
- Setting up MongoDB
- CRUD operations on MongoDB using Beanie

Technical requirements

To follow along, the MongoDB database component is required. The installation procedures for your operating system can be found in their official documentation. The code used in this chapter can be found at <https://github.com/PacktPublishing/Building-Python-Web-APIs-with-FastAPI/tree/main/ch06/planner>.

Setting up SQLAlchemy

The first step to integrate a SQL database into our planner application is to install the SQLAlchemy library. The SQLAlchemy library was built by the creator of FastAPI and is powered by Pydantic and SQLAlchemy. Support from Pydantic will make it easy for us to define models, as we learned in *Chapter 3, Response Models and Error Handling*.

Since we'll be implementing both SQL and NoSQL databases, we'll create a new GitHub branch for this section. In your terminal, navigate to the project directory, initialize a GitHub repository, and commit the existing files:

```
$ git init
$ git add database models routes main.py
$ git commit -m "Committing bare application without a
database"
```

Next, create a new branch:

```
$ git checkout -b planner-sql
```

Now, we are ready to set up SQLAlchemy in our application. From your terminal, activate the virtual environment and install the SQLAlchemy library:

```
$ source venv/bin/activate
(venv)$ pip install sqlalchemy
```

Before diving into adding a database to our planner application, let's look at some of the methods contained in SQLAlchemy that we'll be using in this chapter.

Tables

A table is essentially an object that contains data stored in a database – for example, events data will be stored in an event table. The table will consist of columns and rows where the data will eventually be stored.

The variables defined in the class will represent the columns by default unless denoted as a field. Let's look at how the event table will be defined:

```
class Event(SQLModel, table=True):
    id: Optional[int] = Field(default=None,
                              primary_key=True)
    title: str
    image: str
    description: str
    location: str
    tags: List[str]
```

What Is a Primary Key?
A primary key is a unique identifier for a record contained in a database table.

A primary key is a unique identifier for a record contained in a database table.

Rows

```
new_event = Event(title="Book Launch",  
                  image="src/fastapi.png",  
                  description="The book launch event will  
be held at Packt HQ, Packt city",
```

```
location="Google Meet",  
tags=["packt", "book"])
```

Next, we create a database transaction using the `Session` class:

```
with Session(engine) as session:  
    session.add(new_event)  
    session.commit()
```

The preceding operation may seem alien to you. Let's look at what the **Session** class is and what it does.

Sessions

A session object handles the interaction from code to a database. It primarily acts as an intermediary in executing operations. The `Session` class takes an argument that is the instance of a SQL engine.

Now that we have learned how tables and rows are created, we will look at how a database is created. Some of the methods of the `session` class we'll be using in this chapter include the following:

- `add()`: This method is responsible for adding a database object to memory pending further operations. In the previous code block, the `new_event` object is added to the session's memory, waiting to be committed into the database by the `commit()` method.
- `commit()`: This method is responsible for flushing transactions present in the session.
- `get()`: This method takes two parameters – the model and the ID of the document requested. This method is used to retrieve a single row from a database.

Now that we know how to create tables, rows, and columns, as well as insert data using the `Session` class, let's move on to creating a database and performing CRUD operations in the next section.

Creating a database

In `SQLModel`, connecting to a database is done via a `SQLAlchemy` engine. The engine is created by the `create_engine()` method, imported from the `SQLModel` library.

The `create_engine()` method takes the database URL as the argument. The database URL is in the form of `sqlite:///database.db` or `sqlite:///database.sqlite`. It also takes an optional argument, `echo`, which when set to `True` prints out the SQL commands carried out when an operation is executed.

However, the `create_engine()` method alone isn't sufficient to create a database file. To create the database file, the `SQLModel.metadata.create_all(engine)` method whose argument is an instance of the `create_engine()` method is invoked, such as the following:

```
database_file = "database.db"
engine = create_engine(database_file, echo=True)
SQLModel.metadata.create_all(engine)
```

The `create_all()` method creates the database as well as the tables defined. It is important to note that the file containing the tables is imported into the file where the database connection takes place.

In our planner application, we perform CRUD operations for events. In the database folder, create the following file:

`connection.py`

In this file, we'll configure the necessary data for the database:

```
(venv)$ touch database/connection.py
```

Now that we have created the database connection file, let's create the functions required to connect our application to the database:

1. We'll start by updating the events model class defined in `models/events.py` to a `SQLModel` table model class:

```
from sqlmodel import JSON, SQLModel, Field, Column
from typing import Optional, List

class Event(SQLModel, table=True):
    id: int = Field(default=None, primary_key=True)
    title: str
    image: str
    description: str
    tags: List[str] = Field(sa_column=Column(JSON))
```

```
location: str

class Config:
    arbitrary_types_allowed = True
    schema_extra = {
        "example": {
            "title": "FastAPI Book Launch",
            "image": "https://linktomyimage.com/image.png",
            "description": "We will be discussing the contents of the FastAPI book in this event. Ensure to come with your own copy to win gifts!",
            "tags": ["python", "fastapi", "book", "launch"],
            "location": "Google Meet"
        }
    }
```

In this code block, we have modified the original model class to become a SQL table class.

2. Let's add another SQLAlchemy class that'll be used as the body type during UPDATE operations:

```
class EventUpdate(SQLModel):
    title: Optional[str]
    image: Optional[str]
    description: Optional[str]
    tags: Optional[List[str]]
    location: Optional[str]

class Config:
    schema_extra = {
        "example": {
            "title": "FastAPI Book Launch",
            "image": "https://linktomyimage.com/image.png",
```

```

        //linktomyimage.com/image.png",
        "description": "We will be discussing
        the contents of the FastAPI book in
        this event. Ensure to come with your
        own copy to win gifts!",
        "tags": ["python", "fastapi", "book",
        "launch"],
        "location": "Google Meet"
    }
}

```

3. Next, let's define the configuration needed to create our database and table in `connection.py`:

```

from sqlmodel import SQLModel, Session, create_engine
from models.events import Event

database_file = "planner.db"
database_connection_string = f"sqlite:/// {database_file}"
connect_args = {"check_same_thread": False}
engine_url = create_engine(database_connection_string,
echo=True, connect_args=connect_args)

def conn():
    SQLModel.metadata.create_all(engine_url)

def get_session():
    with Session(engine_url) as session:
        yield session

```

In this code block, we start by defining the dependencies as well as importing the table model class. Next, we create the variable holding the location of the database file (which will be created if it doesn't exist), the connection string, and an instance of the SQL database created. In the `conn()` function, we instruct `SQLModel` to create the database as well as the table present in the file, `Events`, and to persist the session in our application, `get_session()` is defined.

4. Next, let's instruct our application to create a database when it is started. Update `main.py` with the following code:

```
from fastapi import FastAPI
from fastapi.responses import RedirectResponse
from database.connection import conn

from routes.users import user_router
from routes.events import event_router

import uvicorn

app = FastAPI()

# Register routes

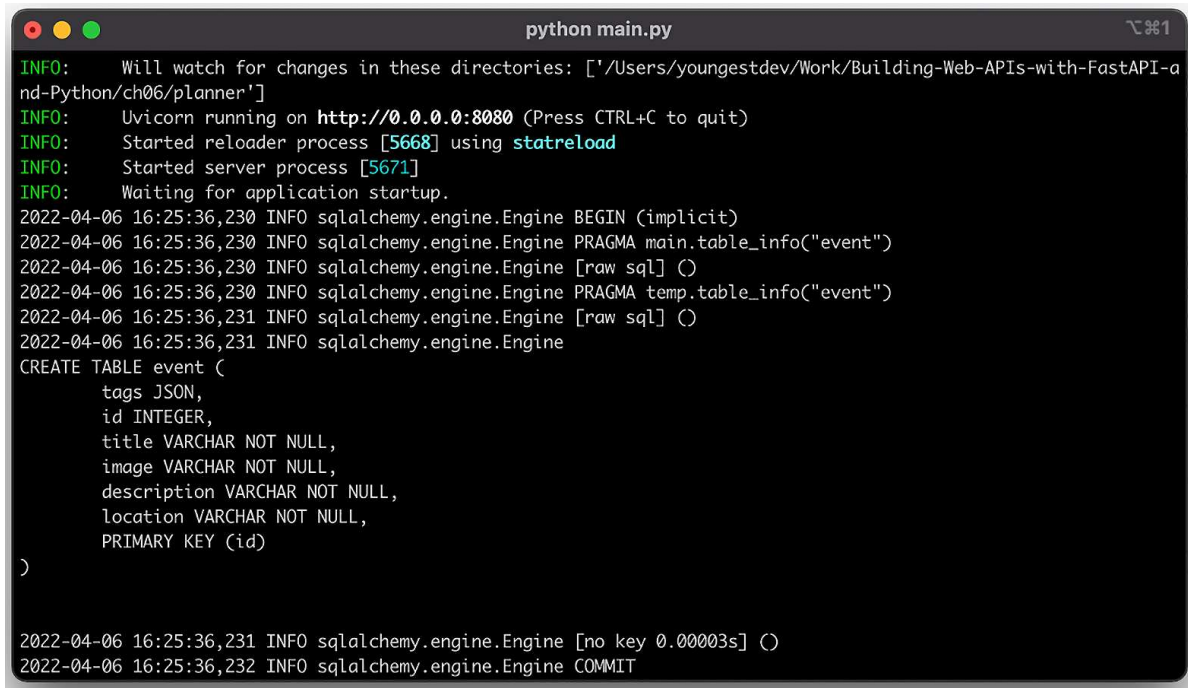
app.include_router(user_router, prefix="/user")
app.include_router(event_router, prefix="/event")

@app.on_event("startup")
def on_startup():
    conn()

@app.get("/")
async def home():
    return RedirectResponse(url="/event/")

if __name__ == '__main__':
    uvicorn.run("main:app", host="0.0.0.0", port=8080,
                reload=True)
```

The database will be created once the application starts. In the startup event, we have called the `conn()` function responsible for creating the database. Start the application in your terminal and you should see the output in your console, indicating that the database has been created as well as the table:



```
python main.py
INFO: Will watch for changes in these directories: ['/Users/youngestdev/Work/Building-Web-APIs-with-FastAPI-and-Python/ch06/planner']
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
INFO: Started reloader process [5668] using statreload
INFO: Started server process [5671]
INFO: Waiting for application startup.
2022-04-06 16:25:36,230 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2022-04-06 16:25:36,230 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("event")
2022-04-06 16:25:36,230 INFO sqlalchemy.engine.Engine [raw sql] ()
2022-04-06 16:25:36,230 INFO sqlalchemy.engine.Engine PRAGMA temp.table_info("event")
2022-04-06 16:25:36,231 INFO sqlalchemy.engine.Engine [raw sql] ()
2022-04-06 16:25:36,231 INFO sqlalchemy.engine.Engine
CREATE TABLE event (
  tags JSON,
  id INTEGER,
  title VARCHAR NOT NULL,
  image VARCHAR NOT NULL,
  description VARCHAR NOT NULL,
  location VARCHAR NOT NULL,
  PRIMARY KEY (id)
)
2022-04-06 16:25:36,231 INFO sqlalchemy.engine.Engine [no key 0.00003s] ()
2022-04-06 16:25:36,232 INFO sqlalchemy.engine.Engine COMMIT
```

Figure 6.1 – The planner database and event table created successfully

The SQL commands displayed in the terminal are there because of setting `echo` to `True` when creating the database engine. Now that we have successfully created the database, let's update our events' CRUD operation routes to use the database.

Creating events

Let's look at the steps:

1. In `routes/events.py`, update the imports to include the `Event` table model class as well as the `get_session()` function. The `get_session()` function is imported so that the routes can access the session object created:

```
from fastapi import APIRouter, Depends, HTTPException, Request, status
from database.connection import get_session
from models.events import Event, EventUpdate
```

What Is Depends?

The Depends class is responsible for exercising dependency injection in FastAPI applications. The Depends class takes a truth source such as a function as an argument and is passed as a function argument in a route, mandating that the dependency condition be satisfied before any operation can be executed.

2. Next, let's update the POST route function responsible for creating a new event, `create_event()`:

```
@event_router.post("/new")
async def create_event(new_event: Event,
                       session=Depends(get_session)) -> dict:
    session.add(new_event)
    session.commit()
    session.refresh(new_event)

    return {
        "message": "Event created successfully"
    }
```

In this code block, we have indicated that the session object required to execute database transactions is dependent on the `get_session()` function we created earlier.

In the function body, the data is added to the session and then committed to the database, after which the database is refreshed.

3. Let's test the routes to preview changes:

```
(venv)$ curl -X 'POST' \
  'http://0.0.0.0:8080/event/new' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "FastAPI Book Launch",
    "image": "fastapi-book.jpeg",
    "description": "We will be discussing the contents
of the FastAPI book in this event. Ensure to come
with your own copy to win gifts!",
```

```

    "tags": [
        "python",
        "fastapi",
        "book",
        "launch"
    ],
    "location": "Google Meet"
}'

```

A successful response is returned:

```

{
    "message": "Event created successfully"
}

```

If the operation failed to execute, an exception will be thrown by the library.

Read events

Let's update the GET route that retrieves the list of events to pull data from the database:

```

@event_router.get("/", response_model=List[Event])
async def retrieve_all_events(session=Depends(get_session)) ->
List[Event]:
    statement = select(Event)
    events = session.exec(statement).all()
    return events

```

Likewise, the route to display an event's data when retrieved by its ID is also updated:

```

@event_router.get("/{id}", response_model=Event)
async def retrieve_event(id: int, session=Depends(get_session))
-> Event:
    event = session.get(Event, id)
    if event:
        return event
    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail="Event with supplied ID does not exist"
    )

```

```
)

raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail="Event with supplied ID does not exist"
)
```

The response model for both routes has been set to the model class. Let's test both routes by first sending a GET request to retrieve the list of the events:

```
(venv)$ curl -X 'GET' \
  'http://0.0.0.0:8080/event/' \
  -H 'accept: application/json'
```

We get a response:

```
[
  {
    "id": 1,
    "title": "FastAPI Book Launch",
    "image": "fastapi-book.jpeg",
    "description": "We will be discussing the contents of
the FastAPI book in this event.Ensure to come with your
own copy to win gifts!",
    "tags": [
      "python",
      "fastapi",
      "book",
      "launch"
    ],
    "location": "Google Meet"
  }
]
```

Next, let's retrieve the event by its ID:

```
(venv)$ curl -X 'GET' \
  'http://0.0.0.0:8080/event/1' \
  -H 'accept: application/json'
```

```
}  
{  
    "id": 1,  
    "title": "FastAPI Book Launch",  
    "image": "fastapi-book.jpeg",  
    "description": "The launch of the FastAPI book will hold  
on xyz.",  
    "tags": [  
        "python",  
        " fastapi"  
    ],  
    "location": "virtual"  
}
```

With the READ operations successfully implemented, let's add an edit feature for our application.

Update events

Let's add the UPDATE route in `routes/events.py`:

```
@event_router.put("/edit/{id}", response_model=Event)  
async def update_event(id: int, new_data: EventUpdate,  
    session=Depends(get_session)) -> Event:
```

In the function body, add the following block of code to retrieve the existing event and handle event changes:

```
event = session.get(Event, id)  
if event:  
    event_data = new_data.dict(exclude_unset=True)  
    for key, value in event_data.items():  
        setattr(event, key, value)  
    session.add(event)  
    session.commit()  
    session.refresh(event)  
  
    return event  
raise HTTPException(
```

```
        status_code=status.HTTP_404_NOT_FOUND,  
        detail="Event with supplied ID does not exist"  
    )
```

In the preceding code block, we check whether an event is present before proceeding to update the event data. Once the event has been updated, the updated data is returned. Let's update the existing article's title:

```
(venv)$ curl -X 'PUT' \  
  'http://0.0.0.0:8080/event/edit/1' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "title": "Packt\'\'s FastAPI book launch II"  
  }'  
  
{  
  "id": 1,  
  "title": "Packt's FastAPI book launch II",  
  "image": "fastapi-book.jpeg",  
  "description": "The launch of the FastAPI book will hold  
on xyz.",  
  "tags": ["python", "fastapi"],  
  "location": "virtual" }
```

Now that we have added the update functionality, let's quickly add a delete operation in the next section.

Delete event

In `events.py`, update the `delete` route defined earlier:

```
@event_router.delete("/delete/{id}")  
async def delete_event(id: int, session=Depends(get_session))  
-> dict:  
    event = session.get(Events, id)  
    if event:  
        session.delete(event)
```

```

        session.commit()

    return {
        "message": "Event deleted successfully"
    }

    raise HTTPException(
        status_code=status.HTTP_404_NOT_FOUND,
        detail="Event with supplied ID does not exist"
    )

```

In this code block, the function checks whether an event whose ID has been supplied exists and then deletes it from the database. Once the operation has been executed, a successful message is returned and an exception thrown if the event doesn't exist. Let's delete the event from the database:

```

(venv)$ curl -X 'DELETE' \
  'http://0.0.0.0:8080/event/delete/1' \
  -H 'accept: application/json'

```

The request returns a successful response:

```

{
  "message": "Event deleted successfully"
}

```

Now, if we retrieve the list of events, we get an empty array for a response:

```

(venv)$ curl -X 'GET' \
  'http://0.0.0.0:8080/event/' \
  -H 'accept: application/json'
[]

```

We have successfully incorporated a SQL database into our application using SQLAlchemy, as well as implementing CRUD operations. Let's commit the changes made to the application before learning how to implement CRUD operations in MongoDB:

```

(venv)$ git add .
(venv)$ git commit -m "[Feature] Incorporate a SQL database and
implement CRUD operations "

```

Switch back to the main branch:

```
(venv)$ git checkout main
```

Now that you are back to the original version of the application, let's incorporate MongoDB as the database platform and implement CRUD operations in the next section.

Setting up MongoDB

There are a number of libraries that allow us to integrate MongoDB into our FastAPI application. However, we'll be using **Beanie**, an asynchronous **Object Document Mapper (ODM)** library, to execute database operations from our application.

Let's install the `beanie` library by running the following command:

```
(venv)$ pip install beanie
```

Before diving into the integration, let's look at some of the methods from the Beanie library and also how database tables are created in this section.

Document

In SQL, the data stored in rows and columns are contained in the table. In a NoSQL database, it is called a document. The document represents how the data will be stored in the database collection. Documents are defined the same way a Pydantic model is defined, except that the `Document` class from the Beanie library is inherited instead.

An example document is defined as follows:

```
from beanie import Document

class Event(Document):
    name: str
    location: str

    class Settings:
        name = "events"
```

The `Settings` subclass is defined to tell the library to create the collection name passed in the MongoDB database.

Now that we know how to create a document, let's look at the methods used for carrying out CRUD operations:

- `.insert()` and `.create()`: The `.insert()` and `.create()` methods are called by the document instance to create a new record in the database. You can also choose to use the `.insert_one()` method to add a singular entry to the database.

To insert many entries into the database, the `.insert_many()` method, which takes a list of the document instance, is called, such as the following:

```
event = Event(name="Packt office launch",
location="Hybrid")
await event.create()
await Event.insert_one(event)
```

- `.find()` and `.get()`: The `.find()` method is used to find a list of documents matching the search criteria passed as the method argument. The `.get()` method is used to retrieve a single document matching the supplied ID. A single document matching a search criterion can be found using the `.find_one()` method, such as the following:

```
event = await Event.get("74478287284ff")
event = await Event.find(Event.location == "Hybrid").to_
list() # Returns a list of matching items
event = await Event.find_one(Event.location == "Hybrid") #
Returns a single event
```

- `.save()`, `.update()`, and `.upsert()`: To update a document, any of these methods can be used. The `.update()` method takes an update query, and the `.upsert()` method is used when a document doesn't match the search criteria. In this chapter, we'll be making use of the `.update()` method. An update query is an instruction followed by the MongoDB database, such as the following:

```
event = await Event.get("74478287284ff")
update_query = {"$set": {"location": "virtual"}}
await event.update(update_query)
```

In this code block, we first retrieve the event and then create an update query to set the location field in the event collection to virtual.

- `.delete()`: This method is responsible for removing a document record from the database, such as the following:

```
event = await Event.get("74478287284ff")
await event.delete()
```

Now that we have learned how the methods contained in the Beanie library work, let's initialize the database in our event planner application, define our documents, and implement the CRUD operations.

Initializing the database

Let's look at the steps to do this:

1. In the database folder, create a `connection.py` file:

```
(venv)$ touch connection.py
```

Pydantic enables us to read environment variables by creating a child class of the `BaseSettings` parent class. When building web APIs, it is a standard practice to store configuration variables in an environment file.

2. In `connection.py`, add the following:

```
from beanie import init_beanie
from motor.motor_asyncio import AsyncIOMotorClient
from typing import Optional
from pydantic import BaseSettings

class Settings(BaseSettings):
    DATABASE_URL: Optional[str] = None

    async def initialize_database(self):
        client = AsyncIOMotorClient(self.DATABASE_URL)
        await init_beanie(
            database=client.get_default_database(),
            document_models=[]
        )

class Config:
    env_file = ".env"
```

In this code block, we start by importing the dependencies required for initializing the database. Then, we define the `Settings` class, which has a `DATABASE_URL` value that is read from the `env_file` environment defined in the `Config` subclass. We also define an `initialize_database` method to initialize the database.

The `init_beanie` method takes the database client, which is the mongo version of the engine created in the `SQLModel` section, and a list of documents.

3. Let's update the model files in the `models` directory to include the MongoDB documents. In `models/events.py`, replace the contents with the following:

```
from beanie import Document
from typing import Optional, List

class Event(Document):
    title: str
    image: str
    description: str
    tags: List[str]
    location: str

class Config:
    schema_extra = {
        "example": {
            "title": "FastAPI Book Launch",
            "image": "https://linktomyimage.com/image.png",
            "description": "We will be discussing the contents of the FastAPI book in this event. Ensure to come with your own copy to win gifts!",
            "tags": ["python", "fastapi", "book", "launch"],
            "location": "Google Meet"
        }
    }
```

```
class Settings:
    name = "events"
```

4. Let's create a Pydantic model for the UPDATE operations:

```
class EventUpdate(BaseModel):
    title: Optional[str]
    image: Optional[str]
    description: Optional[str]
    tags: Optional[List[str]]
    location: Optional[str]

class Config:
    schema_extra = {
        "example": {
            "title": "FastAPI Book Launch",
            "image": "https://linktomyimage.com/image.png",
            "description": "We will be discussing the contents of the FastAPI book in this event. Ensure to come with your own copy to win gifts!",
            "tags": ["python", "fastapi", "book", "launch"],
            "location": "Google Meet"
        }
    }
```

5. In `model/users.py`, replace the content of the file with the following:

```
from typing import Optional, List
from beanie import Document, Link

from pydantic import BaseModel, EmailStr

from models.events import Event
```

```

class User(Document):
    email: EmailStr
    password: str
    events: Optional[List[Link[Event]]]

class Settings:
    name = "users"

class Config:
    schema_extra = {
        "example": {
            "email": "fastapi@packt.com",
            "password": "strong!!!",
            "events": [],
        }
    }

class UserSignIn(BaseModel):
    email: EmailStr
    password: str

```

6. Now that we have defined the documents, let's update the `document_models` field in `connection.py`:

```

from models.users import User
from models.events import Event

async def initialize_database(self):
    client = AsyncIOMotorClient(self.DATABASE_URL)
    await init_beanie(
        database=client.get_default_database(),
        document_models=[Event, User])

```

7. Lastly, let's create an environment file, `.env`, and add the database URL to finalize the database initialization stage:

```

(venv)$ touch .env
(venv)$ echo DATABASE_URL=mongodb://localhost:27017/
planner >> .env

```

Now that we have successfully added the blocks of code to initialize the database, let's proceed to implement the methods for CRUD operations.

CRUD operations

In `connection.py`, create a new `Database` class that takes a model as an argument during initialization:

```
from pydantic import BaseSettings, BaseModel
from typing import Any, List, Optional

class Database:
    def __init__(self, model):
        self.model = model
```

The model passed during initialization is either the `Event` or `User` document model class.

Create

Let's create a method under the `Database` class to add a record to the database collection:

```
async def save(self, document) -> None:
    await document.create()
    return
```

In this code block, we have defined the `save` method to take the document, which will be an instance of the document passed to the `Database` instance at the point of instantiation.

Read

Let's create the methods to retrieve a database record or all the records present in the database collection:

```
async def get(self, id: PydanticObjectId) -> Any:
    doc = await self.model.get(id)
    if doc:
        return doc
```

```

        return False

    async def get_all(self) -> List[Any]:
        docs = await self.model.find_all().to_list()
        return docs

```

The first method, `get()`, takes an ID as the method argument and returns a corresponding record from the database, while the `get_all()` method takes no argument and returns a list of all the records present in the database.

Update

Let's create the method to handle the process of updating an existing record:

```

    async def update(self, id: PydanticObjectId, body:
BaseModel) -> Any:
        doc_id = id
        des_body = body.dict()
        des_body = {k:v for k,v in des_body.items() if v is
not None}
        update_query = {"$set": {
            field: value for field, value in
            des_body.items()
        }}

        doc = await self.get(doc_id)
        if not doc:
            return False
        await doc.update(update_query)
        return doc

```

In this code block, the `update` method takes an ID and the Pydantic schema responsible, which will contain the fields updated from the PUT request sent by the client. The updated request body is first parsed into a dictionary and then filtered to remove None values. Once this has been done, it is then inserted into an update query, which is finally executed by Beanie's `update()` method.

Delete

Lastly, let's create a method to delete a record from the database:

```
async def delete(self, id: PydanticObjectId) -> bool:
    doc = await self.get(id)
    if not doc:
        return False
    await doc.delete()
    return True
```

In this code block, the method checks whether such a record exists before proceeding to delete it from the database.

Now that we have populated our database file with the necessary methods needed to carry out CRUD operations, let's update the routes as well.

routes/events.py

Let's start by updating the imports and creating a database instance:

```
from beanie import PydanticObjectId
from fastapi import APIRouter, HTTPException, status
from database.connection import Database

from models.events import Event
from typing import List
event_database = Database(Event)
```

With the imports and database instance in place, let's update all the routes. Start by updating the GET routes:

```
@event_router.get("/", response_model=List[Event])
async def retrieve_all_events() -> List[Event]:
    events = await event_database.get_all()
    return events

@event_router.get("/{id}", response_model=Event)
async def retrieve_event(id: PydanticObjectId) -> Event:
    event = await event_database.get(id)
    if not event:
```

```
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Event with supplied ID does not exist"
        )
    return event
```

In the GET routes, we are invoking the methods we defined in the database file earlier. Let's update the POST routes:

```
@event_router.post("/new")
async def create_event(body: Event) -> dict:
    await event_database.save(body)
    return {
        "message": "Event created successfully"
    }
```

Let's create the UPDATE route:

```
@event_router.put("/{id}", response_model=Event)
async def update_event(id: PydanticObjectId, body: EventUpdate)
-> Event:
    updated_event = await event_database.update(id, body)
    if not updated_event:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Event with supplied ID does not exist"
        )
    return updated_event
```

Lastly, let's update the DELETE route:

```
@event_router.delete("/{id}")
async def delete_event(id: PydanticObjectId) -> dict:
    event = await event_database.delete(id)
    if not event:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Event with supplied ID does not exist"
        )
```

```
    return {  
        "message": "Event deleted successfully."  
    }
```

Now that we have implemented the CRUD operations to our event routes, let's implement the routes for signing a user up and signing a user in.

routes/users.py

Let's start by updating the imports and creating a database instance:

```
from fastapi import APIRouter, HTTPException, status  
from database.connection import Database  
  
from models.users import User, UserSignIn  
  
user_router = APIRouter(  
    tags=["User"],  
)  
  
user_database = Database(User)
```

Next, update the POST route for signing new users:

```
@user_router.post("/signup")  
async def sign_user_up(user: User) -> dict:  
    user_exist = await User.find_one(User.email ==  
    user.email)  
    if user_exist:  
        raise HTTPException(  
            status_code=status.HTTP_409_CONFLICT,  
            detail="User with email provided exists  
            already."  
        )  
    await user_database.save(user)  
    return {  
        "message": "User created successfully"  
    }
```

In this code block, we check whether such a user with the email passed exists before adding them to the database. Let's add the route to sign users in:

```
@user_router.post("/signin")
async def sign_user_in(user: UserSignIn) -> dict:
    user_exist = await User.find_one(User.email ==
    user.email)
    if not user_exist:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="User with email does not exist."
        )
    if user_exist.password == user.password:
        return {
            "message": "User signed in successfully."
        }
    raise HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Invalid details passed."
    )
```

In this defined route, we first check whether the user exists before checking the validity of their credentials. The method of authentication used here is basic and *not recommended* in production. We'll take a look at proper authentication procedures in the next chapter.

Now that we have implemented the routes, let's start a MongoDB instance as well as our application. Create a folder to house our MongoDB database and start the MongoDB instance:

```
(venv)$ mkdir store
(venv)$ mongod --dbpath store
```

Next, in another window, start the application:

```
(venv)$ python main.py
INFO:      Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C
to quit)
INFO:      Started reloader process [3744] using statreload
INFO:      Started server process [3747]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

Let's test the event routes:

1. Create an event:

```
(venv)$ curl -X 'POST' \
  'http://0.0.0.0:8080/event/new' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "FastAPI Book Launch",
    "image": "https://linktomyimage.com/image.png",
    "description": "We will be discussing the contents
of the FastAPI book in this event. Ensure to come
with your own copy to win gifts!",
    "tags": [
      "python",
      "fastapi",
      "book",
      "launch"
    ],
    "location": "Google Meet"
  }'
```

Here is the response from the preceding operation:

```
{
  "message": "Event created successfully"
}
```

2. Get all events:

```
(venv)$ curl -X 'GET' \  
  'http://0.0.0.0:8080/event/' \  
  -H 'accept: application/json'
```

The preceding request returns a list of events:

```
[  
  {  
    "_id": "624daab1585059e8a3fa77ac",  
    "title": "FastAPI Book Launch",  
    "image": "https://linktomyimage.com/image.png",  
    "description": "We will be discussing the contents  
of the FastAPI book in this event. Ensure to come  
with your own copy to win gifts!",  
    "tags": [  
      "python",  
      "fastapi",  
      "book",  
      "launch"  
    ],  
    "location": "Google Meet"  
  }  
]
```

3. Get an event:

```
(venv)$ curl -X 'GET' \  
  'http://0.0.0.0:8080/event/624daab1585059e8a3fa77ac' \  
  -H 'accept: application/json'
```

This operation returns the event that matches the supplied ID:

```
{  
  "_id": "624daab1585059e8a3fa77ac",  
  "title": "FastAPI Book Launch",  
  "image": "https://linktomyimage.com/image.png",  
  "description": "We will be discussing the contents  
of the FastAPI book in this event. Ensure to come  
with your own copy to win gifts!",  
}
```

```
    "tags": [
        "python",
        "fastapi",
        "book",
        "launch"
    ],
    "location": "Google Meet"
}
```

4. Let's update the event location to Hybrid:

```
(venv)$ curl -X 'PUT' \
    'http://0.0.0.0:8080/event/624daab1585059e8a3fa77ac' \
    \
    -H 'accept: application/json' \
    -H 'Content-Type: application/json' \
    -d '{
    "location": "Hybrid"
}'

{
  "_id": "624daab1585059e8a3fa77ac",
  "title": "FastAPI Book Launch",
  "image": "https://linktomyimage.com/image.png",
  "description": "We will be discussing the contents
of the FastAPI book in this event. Ensure to come
with your own copy to win gifts!",
  "tags": [
    "python", "fastapi",
    "book",
    "launch"
  ],
  "location": "Hybrid"
}
```

5. Lastly, let's delete the event:

```
(venv)$ curl -X 'DELETE' \  
  'http://0.0.0.0:8080/event/624daab1585059e8a3fa77ac' \  
 \  
  -H 'accept: application/json'
```

Here is the response received to the request:

```
{  
  "message": "Event deleted successfully."  
}
```

6. Now that we have tested the routes for the events, let's create a new user and then sign in:

```
(venv)$ curl -X 'POST' \  
  'http://0.0.0.0:8080/user/signup' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "email": "fastapi@packt.com",  
    "password": "strong!!!",  
    "events": []  
  }'
```

The request returns a response:

```
{  
  "message": "User created successfully"  
}
```

Running the request again returns an HTTP 409 error, indicating a conflict:

```
{  
  "detail": "User with email provided exists already."  
}
```

We originally designed the route to check for existing users to avoid duplicates.

7. Now, let's send a POST request to sign in the user we just created:

```
(venv)$ curl -X 'POST' \  
  'http://0.0.0.0:8080/user/signin' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "email": "fastapi@packt.com",  
    "password": "strong!!!"  
  }'
```

The request returns an HTTP 200 success message:

```
{  
  "message": "User signed in successfully."  
}
```

We have successfully implemented CRUD operations using the Beanie library.

Summary

In this chapter, we learned how to add SQL and NoSQL databases using SQLAlchemy and Beanie respectively. We made use of all our knowledge from the previous chapters. We also tested the routes to ensure that they are working as planned.

In the next chapter, you will be introduced to securing your application. You will first be taught the basics of authentication as well as the various authentication methods available to FastAPI developers. You will then implement an authentication system that relies on **JSON Web Token (JWT)** and secure the routes to create, update, and delete events. Lastly, you will modify the route, to create events to allow the linking of events to a user.